

TP1-ALSDD

Gestion d'une population sous forme de listes chaînées

1.Introduction

❖ Objectif

Ce projet vise à implémenter un système de gestion d'une population en utilisant des listes chaînées en C. Il offre plusieurs fonctionnalités, notamment l'ajout, la suppression, la recherche et le tri des familles, tout en garantissant une manipulation efficace des données à l'aide de pointeurs.

❖ Description Générale

La gestion de la population est réalisée à deux niveaux :

1. Liste simple : Contient uniquement les noms des familles, triés alphabétiquement.
2. Liste avancée : Inclut des détails supplémentaires comme les parents, les enfants et les voitures associées.

Le programme est une application de gestion de populations et de familles, divisée en deux parties principales (Partie 1 et Partie 2). Voici les principales fonctionnalités du programme :

- **Partie 1 : Gestion des familles (structure simple)**

1. **Création de populations :**
 - Création manuelle, automatique (aléatoire) ou à partir d'un fichier.
2. **Affichage :**
 - Afficher la liste des familles dans une population.
3. **Tri :**
 - Trier les familles par ordre alphabétique.
4. **Recherche :**
 - Vérifier si une famille existe dans la population.
5. **Insertion/Suppression :**
 - Ajouter ou supprimer des familles.
6. **Fusion :**
 - Fusionner deux populations triées.
7. **Statistiques :**
 - Calculer le nombre de familles dans une population.

- **Partie 2 : Gestion avancée des familles (structure complexe)**

1. **Création de populations :**

- Création manuelle, automatique (aléatoire) ou à partir d'un fichier.

2. **Affichage :**

- Afficher les détails d'une famille (nom, parents, enfants, voitures).

3. **Statistiques :**

- Nombre de familles sans parents.
- Nombre d'enfants dans une population ou une famille spécifique.

4. **Modification :**

- Ajouter/supprimer des parents, enfants ou voitures.
- Insérer un nouveau-né dans une famille.

5. **Recherche :**

- Trouver la famille possédant une voiture spécifique.

6. **Fusion/Déménagement :**

- Fusionner deux populations.
- Déplacer une famille d'une population à une autre.

7. **Exportation :**

- Générer des rapports (statistiques, comparaisons, actes de naissance, vols de voitures).

❖ **Fonctionnalités générales**

- **Interface utilisateur interactive :**

- Menus clairs avec navigation au clavier/souris.
- Couleurs et mise en forme pour une meilleure lisibilité.

- **Gestion de la mémoire :**

- Allocation et libération dynamique des structures de données.

- **Import/Export :**

- Charger des données depuis un fichier.
- Exporter des rapports dans des fichiers texte.

Autres fonctionnalités

- **Confirmation des actions** : Demandes de confirmation pour les opérations critiques.
- **Gestion des erreurs** : Messages d'erreur clairs pour les opérations invalides.

Ce programme offre une solution complète pour gérer et analyser des populations de familles, avec une interface conviviale et des fonctionnalités avancées de manipulation de données.

❖ Contraintes Techniques

Le projet repose uniquement sur le langage C, en exploitant les pointeurs pour la gestion des listes chaînées. Il respecte les principes suivants :

- Gestion dynamique de la mémoire pour optimiser l'utilisation des ressources.
- Maintien de l'ordre alphabétique pour assurer un accès rapide et structuré aux données.
- Mise en place d'algorithmes efficaces pour les opérations courantes sur les listes.

❖ Équipe de Développement

- **Merzougui Mohammed Abdeldjalil**
- **Ladjouzi Imene**
- **Sous la supervision de : *Kermi Adel***

❖ Public Cible

Ce projet est destiné principalement à notre professeur, ainsi qu'à nos camarades de classe et aux développeurs intéressés par la gestion des structures de données en C.

❖ Technologies et Bibliothèques Utilisées

Le projet est développé en **langage C** et repose sur plusieurs bibliothèques standard et spécifiques pour assurer une gestion efficace des listes chaînées et des fichiers.

<stdio.h>	Gestion des entrées/sorties (printf, scanf, fopen, fclose, etc.).
<stdlib.h>	Allocation dynamique de mémoire (malloc, free), conversions de types.
<stdbool.h>	Manipulation des valeurs booléennes (true, false).
<time.h>	Gestion du temps et des dates (time, srand, etc.).

<string.h>	Opérations sur les chaînes de caractères (strcmp, strcpy, strcat, etc.).
<ctype.h>	Manipulation des caractères (toupper, tolower, isdigit, etc.).
<windows.h>	Fonctions spécifiques à Windows (couleurs, gestion de la console, Sleep, etc.).
<conio.h>	Gestion des entrées clavier sans appuyer sur "Entrée" (_getch, _kbhit).
<unistd.h>	Fonctions pour la gestion des processus et des délais (sleep).
<locale.h>	Gestion de la localisation et des encodages de caractères.

❖ Manipulation des fichiers :

Le projet inclut des fonctionnalités pour charger et sauvegarder les données des familles dans des fichiers. Pour cela, on utilise les fonctions de <stdio.h> :

- fopen() et fclose() : ouvrir et fermer un fichier.
- fprintf() et fscanf() : écrire et lire des données formatées.
- fread() et fwrite() : lecture et écriture en mode binaire.

2.Installation et Configuration

❖ Compatibilité

Ce programme est **exclusivement compatible avec Windows** car il utilise l'API Windows pour la gestion des entrées utilisateur, notamment la souris.

❖ Navigation Intuitive

L'interface du menu est fluide et réactive. Elle prend en charge :

- **La souris** : navigation et sélection des options par un simple **clic gauche**.
- **Le clavier** : navigation à l'aide des **flèches directionnelles** et validation avec **Entrée**.

Cette intégration assure une expérience utilisateur ergonomique et fluide.

❖ Installation

Aucune installation complexe n'est requise. Suivez ces étapes pour exécuter le programme :

1. Télécharger et extraire le dossier du projet.
2. Ouvrir un terminal dans le dossier du projet.

3. Compiler le programme avec un compilateur compatible (ex : MinGW) :

`gcc main.c -o gestion_population.exe -Wall`

4. Exécuter le fichier compilé :

`gestion_population.exe` (“si le nom de votre fichier est `gestion_population`”)

❖ Pré-requis

- **Windows** (le programme ne fonctionne pas sur Linux/macOS).
- **Un compilateur C** compatible avec Windows (ex: MinGW, MSVC).

3. Organisation des fichiers

le projet est divisé en 7 fichiers principaux :

a) `part1.h` et `part1.c`

- **Rôle** : Gèrent la Partie 1 du projet (gestion simplifiée des familles).
- **Contenu** :
 - **`part1.h`** : Définit les structures (`famille1`) et déclare les fonctions pour manipuler les listes chaînées de familles.
 - **`part1.c`** : Implémente les fonctions déclarées dans `part1.h` (création, tri, fusion, etc.).

b) `part2.h` et `part2.c`

- **Rôle** : Gèrent la Partie 2 (gestion avancée avec parents, enfants, voitures).
- **Contenu** :
 - **`part2.h`** : Définit des structures complexes (famille, personne, voiture) et leurs fonctions.
 - **`part2.c`** : Implémente des opérations avancées (déménagement, naissance, recherche de voitures volées, etc.).

c) `table.h` et `table.c`

- **Rôle** : Stockent des données statiques (noms, prénoms, marques de voitures) pour la génération aléatoire.
- **Contenu** :
 - `table.h` : Déclare des tableaux de chaînes (ex: `noms_familles`, `prenoms_enfants`).
 - `table.c` : Définit ces tableaux pour être utilisés dans `part1.c` et `part2.c`.

d) `main.c`

- **Rôle** : Point d'entrée du programme, gère les menus et l'interaction utilisateur.
- **Contenu** :
 - Interfaces graphiques textuelles avec couleurs.
 - Détection des touches clavier/souris.
 - Appel des fonctions de part1.c ou part2.c selon les choix de l'utilisateur.

4. Guide d'Utilisation - FAQ

❖ Généralités

Q1 : Que fait ce programme ?

Ce programme permet de gérer des populations de familles avec deux modes :

- Partie 1 : Gestion simple de listes de noms de familles (tri, fusion, recherche).
- Partie 2 : Gestion détaillée avec parents, enfants, voitures et statistiques.

Q2 : Comment lancer le programme ?

Compilez avec :

```
gcc main.c part1.c part2.c table.c -o gestion_familles
```

```
./gestion_familles
```

❖ Partie 1 - Gestion Simple

Q3 : Comment créer une liste de familles ?

Dans le menu Partie 1 :

1. Choisissez "Créer une population"
2. Sélectionnez le mode :
 - Manuel (saisie des noms)
 - Aléatoire (génération automatique)
 - Fichier (import depuis un fichier texte)

Q4 : Comment trier les familles ?

Sélectionnez "Trier la population" dans le menu. Le programme les classe par ordre alphabétique.

Q5 : Puis-je fusionner deux listes ?

Oui, utilisez l'option "Fusionner 2 populations" et choisissez où stocker le résultat.

❖ Partie 2 - Gestion Avancée

Q6 : Comment ajouter un enfant à une famille ?

1. Allez dans "Modifier une famille"
2. Choisissez "Insérer un enfant"
3. Entrez le prénom et le sexe (M/F)

Q7 : Comment trouver qui possède une voiture ?

Utilisez "Rechercher une voiture" et entrez son numéro de matricule. Un rapport s'affichera.

Q8 : Puis-je exporter des données ?

Oui, dans "Exporter un fichier", vous pouvez générer :

- Des statistiques sur la population
- Un acte de naissance pour un enfant
- Un rapport de vol de voiture

❖ Problèmes Courants

Q9 : Pourquoi je ne trouve pas une famille ?

- Vérifiez l'orthographe du nom
- Assurez-vous que la population existe bien (créez-la si nécessaire)

Q10 : Que faire si le programme ne répond plus ?

- Fermez le terminal et relancez le programme
- Vérifiez que vous n'avez pas saisi de caractères invalides

❖ Trucs et Astuces

Q11 : Comment naviguer plus vite dans les menus ?

- Utilisez les flèches ↑/↓ pour vous déplacer
- Appuyez sur Entrée pour valider
- Échap permet de revenir en arrière

Q12 : Où sont sauvegardés les fichiers exportés ?

Les rapports sont créés dans le même dossier que l'exécutable du programme.

Q13 : Comment quitter proprement ?

Sélectionnez toujours "Quitter" dans les menus plutôt que de fermer la fenêtre.

Besoin d'aide supplémentaire ?

Consultez les commentaires dans le code source ou contactez les développeurs.

4. Documentation de Code :

table.h && table.c

1. Rôle

Ce fichier d'en-tête (table.h) déclare des **données externes** utilisées comme bases de référence pour générer aléatoirement des noms de familles, des prénoms, des marques de voitures et des matricules. Ces tableaux sont définis dans table.c et utilisés principalement dans les parties 1 et 2 pour peupler les structures de données.

2. Variables Externes

Variable	Type	Description
names	ch20	Tableau de noms de familles (ex: "Dupont", "Martin").
taille	int	Taille du tableau names.
prenomH	ch20	Tableau de prénoms masculins (ex: "Jean", "Paul").
sizeH	int	Taille du tableau prenomH.
prenomF	ch20	Tableau de prénoms féminins (ex: "Marie", "Sophie").
sizeF	int	Taille du tableau prenomF.
marques	ch20	Tableau de marques de voitures (ex: "Toyota", "Renault").
size_marque	int	Taille du tableau marques.
matricules	Ch20	Tableau de plaques d'immatriculation (ex: "AB-123-CD", "XY-789-ZZ").
size_mat	int	Taille du tableau matricules.

Remarque : `ch20` ⇔ `char[20]`

3. Utilisation Typique

Ces variables sont utilisées pour :

- **Génération aléatoire :**
 - Dans `part1.c` : Création de noms de familles via `names`.
 - Dans `part2.c` : Attribution de prénoms (`prenomH/prenomF`), marques et matricules aux familles.
 - **Initialisation :**

Les tailles (`taille`, `sizeH`, etc.) permettent d'éviter les débordements lors de l'accès aux tableaux.
-

4. Dépendances

- **Fichiers associés :**
 - `table.c` : Définit le contenu des tableaux déclarés ici.
 - `part1.c` / `part2.c` : Utilisent ces données pour remplir les structures.
 - **Modules concernés :**
 - Fonctions de création aléatoire (ex: `creer_pop` dans `part1.c`).
-

5. Points Clés

- **Modularité :**

Séparation claire entre la déclaration (`table.h`) et la définition (`table.c`).
 - **Extensibilité :**

Pour ajouter de nouveaux noms/prénoms, il suffit de modifier `table.c` sans toucher aux autres fichiers.
 - **Sécurité :**

Les tailles (`sizeH`, `sizeF`, etc.) sont essentielles pour itérer sans risques sur les tableaux.
-

6. Exemple d'Accès



```
1  #include "table.h"
2
3  // Obtenir un prénom masculin aléatoire
4  char* random_prenomH = prenomH[rand() % sizeH];
```

Méthodologie de Collecte des Données

1. Origine des Données

Les données utilisées proviennent principalement des comptes mail étudiants de l'ESI (École Nationale Supérieure d'Informatique). Cette source fournit une base de noms et prénoms algériens authentiques.

2. Processus de Collecte

Phase d'Extraction

- Accès aux données via le système d'activation des comptes mail étudiants
- Extraction des listes complètes contenant noms et prénoms
- Stockage initial dans des fichiers texte bruts

Traitement des Données

- Utilisation combinée d'outils Linux et de scripts Python pour :
 - Isoler les noms de famille
 - Éliminer les doublons
 - Uniformiser le formatage
- Séparation manuelle des prénoms par genre
- Création de deux listes distinctes pour prénoms masculins et féminins

Génération des Données Complémentaires

- Recherche documentaire pour les marques de véhicules

- Développement d'un générateur de plaques d'immatriculation
- Vérification manuelle des résultats

3. Intégration Technique

Les données traitées sont organisées en tableaux C statiques dans le fichier table.c, avec :

- Un tableau principal pour les noms de famille
- Deux tableaux distincts pour les prénoms (masculin/féminin)
- Un tableau pour les marques automobiles
- Un tableau pour les immatriculations

4. Mécanisme de Sélection

Le système utilise une sélection aléatoire indexée pour :

- Choisir entre prénoms masculins et féminins
- Piocher uniformément dans les différentes listes
- Garantir une distribution équilibrée des données

5. Avantages de l'Approche

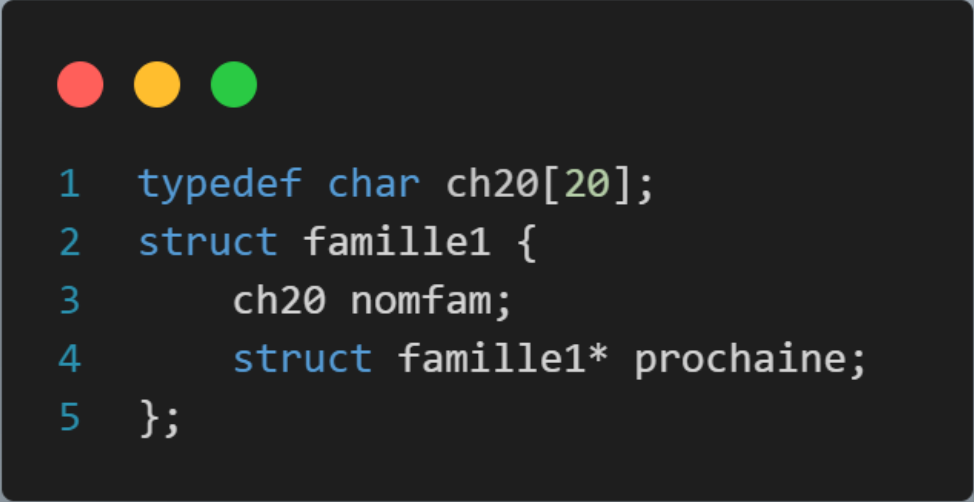
- Authenticité des données nominatives
- Cohérence culturelle et régionale
- Processus reproductible
- Maintenance aisée des listes

Part1.c && part1.h

Gestion des Familles (simple)

❖ Structure de Données

- Définition de la Structure famille1



```
1 typedef char ch20[20];
2 struct famille1 {
3     ch20 nomfam;
4     struct famille1* prochaine;
5 };
```

- nomfam : Stocke le nom de la famille (chaîne de caractères de 20 caractères maximum).
- prochaine : Pointeur vers le prochain élément de la liste.

❖ Fonctions Principales

- Allocation et Libération de Mémoire

void allouer(struct famille1 **p);

- Utilise malloc pour allouer dynamiquement une nouvelle structure famille1.
- Initialise prochaine à NULL.

void liberer(struct famille1 *p);

- Parcourt la liste et libère chaque élément avec free.

- Empêche les fuites mémoire en mettant les pointeurs à NULL après libération.

Manipulation des Noms et Adresses

void aff_nom(struct famille1 *pop, char *nom);

- Copie la chaîne nom dans nomfam de la structure pop avec strcpy.

void aff_adresse(struct famille1 *head_dest, struct famille1 *head_source);

- Affecte l'adresse head_source au champ prochaine de head_dest.

char* nom_fam(struct famille1 *pop);

- Retourne un pointeur vers nomfam.

struct famille1 *proc(struct famille1 *pop);

- Retourne l'adresse du maillon suivant en renvoyant pop->prochaine.

Recherche et Vérifications

bool present_nonTrie(struct famille1 *pop, char *nom);

- Parcourt la liste avec une boucle while.
- Compare nomfam avec strcmp.
- Retourne true si trouvé, sinon false.

bool existe(int *arr, int size, int val);

- Recherche séquentiellement val dans arr.

struct famille1* present(struct famille1 *pop, char *nom);

- Fonction similaire à present_nonTrie mais retourne l'adresse du nœud trouvé.

Création et Modification de la Liste

struct famille1 *creer_pop(int N);

- Utilise une boucle for pour insérer N noms générés aléatoirement.
- Appelle malloc et strcpy pour chaque nouvel élément.

struct famille1* creer_pop_manuel(int n);

- Demande n noms à l'utilisateur via scanf.

- Insère chaque nom manuellement dans la liste.

void inser(struct famille1 **pop, char *nom);

- Vérifie si le nom existe déjà avec present.
- Alloue un nouvel élément avec malloc et l'insère en tête de liste.

void supp(struct famille1 pop, char* nom);**

- Recherche le nœud correspondant avec present.
- Ajuste les pointeurs prochaine pour exclure le nœud trouvé.
- Libère l'espace mémoire avec free.

Tri et Fusion

bool ordre_alpha(char chaine1[], char chaine2[], int pos);

- Utilise strcmp pour comparer les chaînes.

void trier_pop(struct famille1* pop);

- Implémente un tri par insertion.
- Réorganise les pointeurs des maillons pour classer les noms par ordre alphabétique.

struct famille1* fusion_pops(struct famille1 *pop1, struct famille1 *pop2);

- Fusionne deux listes triées en une seule.
- Utilise une approche de fusion similaire au tri fusion.

Affichage et Gestion Avancée

void afficher_pop1(struct famille1 *pop);

- Parcourt la liste avec une boucle while et affiche nomfam avec printf.

void inser_n_fam_m(struct famille1 **fam, int n);

- Demande n noms à l'utilisateur et insère chaque famille dans la liste triée.

void inser_n_fam_a(struct famille1* fam,int n);

- Génère n noms aléatoires et les insère automatiquement.

struct famille1* copier_liste(struct famille1 *original);

- Alloue dynamiquement une nouvelle liste avec malloc.
- Copie chaque élément de original en conservant l'ordre.

Gestion des Fichiers

struct famille1* creer_fichier();

- Lit un fichier contenant des noms séparés par /.
- Crée une liste chaînée à partir des données du fichier.

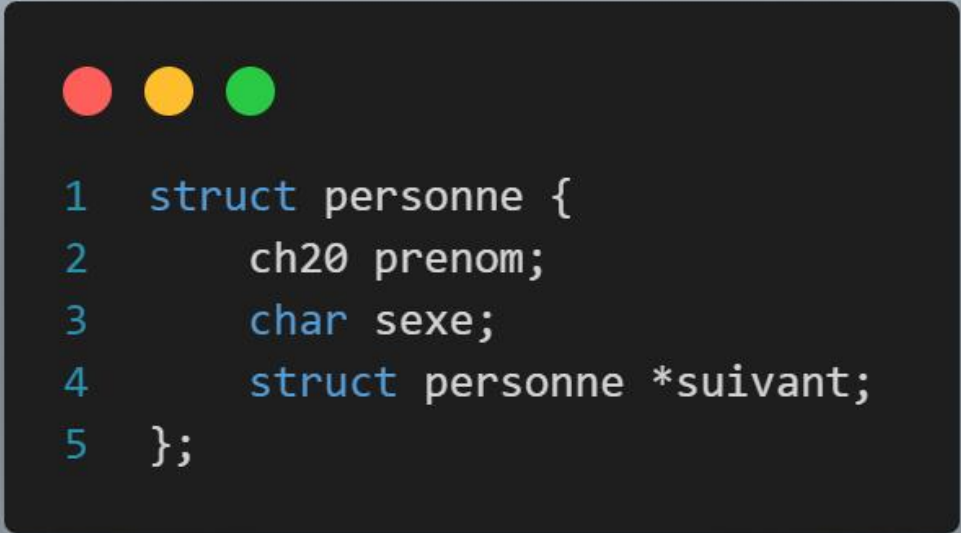
char * eff_espace(char * word);

- Supprime les espaces en début et fin de chaîne avec isspace et strlen.

Part2.c && part2.h

❖ Structures de Données

- Structure `personne`



```
1 struct personne {
2     ch20 prenom;
3     char sexe;
4     struct personne *suivant;
5 };
```

- Structure voiture



```
1 struct voiture {
2     ch20 marque;
3     ch20 numero;
4     struct voiture *suivant;
5 }
```

- Structure famille



```
1 struct famille {
2     ch20 nom;
3     pp parent;
4     pp enfant;
5     pv voiture;
6     struct famille *suivant;
7 };
```


- Structure nouvelle_fam



```
1  typedef struct nouvelle_fam {
2      ch20 nom_fam;
3      pp parents;
4      pp enfants;
5      pv voitures;
6      pv voitures_volees;
7      struct nouvelle_fam *suivant;
8  } *pnf;
```

- Définitions des Macros pour l’Affichage



```
1  #define RED "\x1B[31m"
2  #define GREEN "\x1B[32m"
3  #define YELLOW "\x1B[33m"
4  #define BLUE "\x1B[34m"
5  #define WHITE "\x1B[37m"
6  #define BLEU "\033[38;5;153m"
7  #define pink "\033[38;5;225m"
8  #define gris "\033[38;5;250m"
9  #define green "\033[38;5;157m"
10 #define lavender "\033[38;5;147m"
11 #define JAUNE "\033[1;33m"
```

Macros ANSI : Ces chaînes de caractères spéciales sont insérées dans les sorties console pour modifier la couleur du texte, ce qui améliore la visibilité et permet de distinguer facilement les différents types de messages (erreurs, succès, avertissements, etc.).

5. Modules Demandés – Fonctions Spécifiques :

Ce segment du code implémente plusieurs fonctions demandées dans le projet, servant à calculer, rechercher et trier les données.

5.1. Calculs et Comptages

- **int nbfamillesansparent(pf pop) :**
 - Parcourt la liste des familles et compte celles qui n'ont pas de parents (c.-à-d. où le champ parent est NULL).
 - Technique utilisée : Boucle while pour itérer sur la liste chaînée.
- **int nbenfant(pf pop) :**
 - Itère sur chaque famille, puis sur la liste d'enfants de chaque famille, pour compter le nombre total d'enfants.
 - Technique utilisée : Double boucle imbriquée sur deux listes chaînées.
- **int nbenfnomfam(pf pop, ch20 nomfam) :**
 - Compare les noms de famille et compte le nombre d'enfants associés à une famille donnée.
 - Technique utilisée : Utilisation de strncasecmp pour une comparaison insensible à la casse.

5.2. Insertion et Recherche

- **void naissanceTrie(pf pop, ch20 nomfam, ch20 prenom, char sexe) :**
 - Permet d'insérer un nouveau-né dans la liste des enfants d'une famille donnée, en vérifiant l'unicité du prénom.
 - Technique utilisée : Vérification avec strncasecmp et insertion à la fin de la liste chaînée.
- **pf RechFamVoit_1(pf head, ch20 numero) :**
 - Recherche dans chaque famille la présence d'une voiture correspondant à un numéro matricule donné.
 - Technique utilisée : Double parcours de liste (famille puis liste de voitures) et comparaison avec strcmp.
- **pnf RechFamVoit_2(pnf pop, ch20 matricule) :**

- Recherche une famille dans une autre liste (probablement une liste de familles avec voitures volées) par matricule.
- Technique utilisée : Parcours imbriqué et utilisation de strcmp pour vérifier les correspondances partielles.
- **void treer_famille(pf head) :**
 - Trie la liste des familles en ordre alphabétique, en échangeant le contenu des structures plutôt que de réarranger les pointeurs.
 - Technique utilisée : Algorithme de tri à bulles modifié pour listes chaînées, allocation temporaire de mémoire pour le swap.

6. Modules Additionnels – Fonctions de Gestion Avancée

Ces fonctions ajoutent des fonctionnalités supplémentaires pour la gestion fine des listes, la saisie manuelle et la manipulation d'entités.

6.1. Fonctions de Vérification et de Recherche dans des Listes de Chaînes

- **bool existe_str(ch20 *arr, int size, ch20 str) :**
 - Parcourt un tableau de chaînes pour vérifier si une chaîne donnée y figure.
 - Technique utilisée : Boucle for et comparaison insensible à la casse avec strcasecmp.
- **bool existe_str_liste(pp par, pp enf, ch20 str) :**
 - Vérifie dans deux listes chaînées (parents et enfants) si un prénom existe déjà.
 - Technique utilisée : Parcours des deux listes et utilisation de strcmp pour assurer l'unicité.
- **bool present_famille_nonTrie(pf pop, char *nom) :**
 - Recherche si une famille existe dans une liste non triée.
 - Technique utilisée : Parcours séquentiel et comparaison avec strcmp.

6.2. Fonctions de Création de Listes (Personnes, Voitures, Familles)

- **pp creer_personne(...) :**
 - Crée une liste chaînée de personnes en générant aléatoirement des prénoms et en assignant un sexe (M/F).
 - Techniques spécifiques :
 - Génération aléatoire avec rand() % sizeH ou rand() % sizeF pour choisir des prénoms.

- Utilisation de tableaux d'indices pour éviter la répétition de prénoms dans la même liste.
- Insertion en fin de liste via chaînage.
- **pv creer_voiture(int N) :**
 - Crée une liste chaînée de voitures en sélectionnant aléatoirement des marques et matricules, avec gestion des indices pour éviter les doublons.
 - Technique utilisée : Allocation dynamique et mise en place d'une structure de suivi des indices déjà utilisés.
- **pf creer_famille(int N) :**
 - Construit une liste de familles en combinant les modules de création de personnes et de voitures.
 - Techniques utilisées :
 - Utilisation d'un tableau pour mémoriser les indices déjà utilisés pour les noms de familles.
 - Appel des fonctions creer_personne et creer_voiture pour remplir les champs de chaque famille.
 - Chaînage des familles par insertion séquentielle et mise à jour du pointeur de tête.
- **pf SimplePop_ToFamille(struct famille1 *pop) :**
 - Convertit une liste de familles de la partie 1 en une liste de familles enrichie de personnes et de voitures.
 - Technique utilisée : Parcours de la liste initiale et création d'une nouvelle structure pour chaque élément avec appel aux fonctions d'allocation et d'affectation.

6.3. Fonctions d'Affichage et de Contrôle Visuel

- **void Color(int couleurDuTexte, int couleurDeFond) :**
 - Change la couleur du texte et du fond dans la console sous Windows en utilisant les fonctions de l'API Windows.
 - Technique utilisée : Appel à SetConsoleTextAttribute après récupération du handle avec GetStdHandle.
- **void print_famille1(pf fam) :**
 - Affiche de manière graphique et structurée le contenu d'une famille, en formatant les noms, parents, enfants et voitures dans des boîtes et lignes.
 - Techniques spécifiques :

- Utilisation de boucles imbriquées pour parcourir chaque liste (parents, enfants, voitures).
- Conversion des chaînes pour respecter une convention de majuscule/minuscule.
- Insertion de séparateurs visuels (lignes d'underscore, tirets) pour délimiter les blocs d'information.
- Utilisation des macros de couleurs pour distinguer les sections (JAUNE, BLEU, GREEN, etc.).

6.4. Fonctions de Saisie Manuelle et d'Insertion

- **pp remplir_parent(int N) et pp remplir_enfant(pp fam, int N) :**
 - Ces fonctions demandent à l'utilisateur de saisir les informations pour les parents et les enfants, en vérifiant l'unicité des prénoms et en validant le sexe (avec une boucle qui impose la saisie de 'M' ou 'F').
 - Techniques utilisées : Saisie via scanf, validation de l'entrée et gestion des sauts de ligne avec getchar.
- **pv remplir_voituer(int N) :**
 - Procède à la saisie manuelle des informations concernant les voitures (marque et matricule), puis construit la liste chaînée correspondante.
- **void reinserer() et void inserer_famille(...) :**
 - Permettent de gérer les cas où une famille existe déjà dans la population, en proposant à l'utilisateur de réessayer ou d'annuler l'insertion.
 - Techniques utilisées : Gestion interactive via _getch() pour capturer des choix sans attendre la validation par "Entrée" et rafraîchissement de l'affichage avec clear_screen() et moveCursorToTop().

6.5. Fonctions de Suppression et de Déplacement

- **void supp_famille(pf *head, char* nom) et void supp_famille_pop(pf *head, char* nom) :**
 - Ces fonctions recherchent une famille dans la liste et la retirent en ajustant les pointeurs de chaînage, tout en libérant correctement la mémoire associée aux listes imbriquées (parents, enfants, voitures).
 - Technique utilisée : Itération jusqu'à trouver l'élément cible, utilisation d'un pointeur précédent pour modifier le chaînage et appel de free().
- **void demenager_famille(pf *pop1, pf *pop2) :**
 - Permet de déplacer une famille d'une population à une autre. La famille est d'abord insérée dans la deuxième liste puis supprimée de la première.

- Technique utilisée : Réutilisation des fonctions d'insertion et de suppression pour garantir l'intégrité des listes après déplacement.
- **Fonctions de suppression sur les listes de voitures et de personnes** (ex. `supprimer_voiture`, `supprimer_enfant`, `supprimer_parent`) :
 - Ces fonctions permettent de retirer un élément spécifique d'une liste chaînée en mettant à jour les liens entre les nœuds et en libérant la mémoire.
 - Technique utilisée : Parcours séquentiel avec comparaison (par `strcmp` ou `strcasecmp`) et gestion des cas particuliers (élément en tête, élément intermédiaire).

6.6. Fonctions de Copie et Fusion

- **`pp copier_personne(pp original)` et `pv copier_voit(pv original)` :**
- Créent une copie exacte d'une liste chaînée de personnes ou de voitures, afin de pouvoir manipuler ou fusionner les données sans altérer l'original.
- Technique utilisée : Allocation de nouvelles cases mémoire pour chaque nœud et chaînage dans le même ordre.
- **`pf copier_pop(pf pop)` :**
- Copie une population entière (liste de familles) en copiant chacune des sous-listes (parents, enfants, voitures) via les fonctions de copie correspondantes.
- **`pf fusion1(pf pop1, pf pop2)` :**
- Fusionne deux populations en les chaînant l'une à la suite de l'autre puis en réordonnant la liste par ordre alphabétique via la fonction de tri.
- Technique utilisée : Parcours jusqu'à la fin de la première liste puis affectation du pointeur du dernier élément à la tête de la seconde liste, suivi d'un tri pour harmoniser l'ordre.

6.7. Fonctions de Création Manuelle

- **`pp creer_personne_manuel(int N, bool parent, int np, pp par)` et `pv creer_voiture_manuel(int N)` et `pf creer_famille_manuel(int N)` :**
- Ces fonctions offrent une alternative à la génération aléatoire en permettant à l'utilisateur de saisir directement les données.
- Technique utilisée : Saisie via `scanf` et `getchar`, gestion des entrées de l'utilisateur, validation des données et création de listes chaînées correspondantes.

7. Conclusion

3.7. Saisie Manuelle et Insertion

Ces fonctions permettent de saisir manuellement les informations sur les familles, parents, enfants et voitures. Contrairement aux fonctions de création aléatoire, elles impliquent une interaction directe avec l'utilisateur via des entrées clavier.

1. pp remplir_parent(int N), pp remplir_enfant(pp fam, int N)

- **Objectif** : Permettre à l'utilisateur de saisir les informations des parents et des enfants.
- **Techniques utilisées** :
 - **Boucle de saisie sécurisée** : Utilisation de scanf et fgets pour récupérer les entrées utilisateur en évitant les débordements.
 - **Validation des entrées** : Vérification de la saisie du sexe (M ou F), avec une boucle qui redemande une saisie valide.
 - **Gestion de l'unicité des prénoms** : Utilisation de strcmp pour comparer les prénoms et s'assurer qu'il n'y a pas de doublon dans la liste.
 - **Insertion en fin de liste chaînée** : Création d'un nouveau nœud et mise à jour du pointeur du dernier élément.

2. pv remplir_voiture(int N)

- **Objectif** : Saisie manuelle des informations relatives aux voitures (marque et matricule).
- **Techniques utilisées** :
 - **Saisie et validation des matricules** : Vérification que chaque matricule est unique avant l'insertion.
 - **Insertion ordonnée dans la liste chaînée** : Les voitures sont ajoutées tout en maintenant l'ordre alphabétique ou numérique.

3. void reinserer(), void inserer_famille(...)

- **Objectif** : Gérer les cas où une famille existe déjà.
 - **Techniques utilisées** :
 - **Gestion interactive des doublons** : L'utilisateur est informé qu'une famille existe déjà et peut choisir de réessayer ou d'annuler.
 - **Utilisation de _getch()** : Permet de capturer les choix utilisateur sans nécessiter d'appui sur "Entrée".
 - **Rafraîchissement de l'affichage** : Utilisation de clear_screen() et moveCursorToTop() pour offrir une meilleure expérience utilisateur.
-

3.8. Suppression et Déplacement

Ces fonctions permettent de gérer dynamiquement les listes chaînées en supprimant des éléments ou en les déplaçant.

1. void supp_famille(pf *head, char* nom)

- **Objectif** : Supprimer une famille de la liste.
- **Techniques utilisées** :
 - **Parcours séquentiel de la liste** pour localiser l'élément à supprimer.
 - **Mise à jour des pointeurs** pour maintenir l'intégrité de la liste.
 - **Libération de mémoire avec free()** pour éviter les fuites mémoire.

2. void demenager_famille(pf *pop1, pf *pop2)

- **Objectif** : Déplacer une famille d'une liste de population à une autre.
- **Techniques utilisées** :
 - **Copie du nœud** dans la nouvelle liste.
 - **Suppression de l'ancienne entrée** pour éviter les doublons.
 - **Réorganisation des pointeurs** pour garantir la cohérence des deux listes.

3. void supprimer_voiture, void supprimer_enfant, void supprimer_parent

- **Objectif** : Supprimer un élément spécifique dans une liste.
- **Techniques utilisées** :
 - **Utilisation de strcmp ou strcasecmp** pour identifier l'élément cible.
 - **Mise à jour des liens chaînés** pour retirer l'élément sans rompre la structure.
 - **Libération de mémoire** après la suppression.

3.9. Copie et Fusion

Ces fonctions permettent de dupliquer des structures ou d'en fusionner plusieurs.

1. pp copier_personne(pp original), pv copier_voit(pv original)

- **Objectif** : Copier une liste chaînée de personnes ou de voitures.
- **Techniques utilisées** :
 - **Allocation dynamique de nouveaux nœuds** pour chaque élément copié.
 - **Parcours de la liste originale** et copie des données champ par champ.

- **Mise à jour des pointeurs** pour recréer la structure sans dépendance avec l'original.

2. pf copier_pop(pf pop)

- **Objectif** : Copier une population entière.
- **Techniques utilisées** :
 - **Recopie récursive des sous-listes** (parents, enfants, voitures).
 - **Préservation de la hiérarchie** pour éviter des pertes de données.

3. pf fusion1(pf pop1, pf pop2)

- **Objectif** : Fusionner deux populations en une seule.
- **Techniques utilisées** :
 - **Concaténation des deux listes** en reliant le dernier élément de pop1 au premier de pop2.
 - **Tri final** pour assurer un ordre cohérent après la fusion.

3.10. Création Manuelle

Ces fonctions permettent de créer des entités en demandant à l'utilisateur d'entrer manuellement les informations, sans génération aléatoire.

1. pp creer_personne_manuel(int N, bool parent, int np, pp par)

- **Objectif** : Créer une liste chaînée de personnes à partir d'une saisie utilisateur.
- **Techniques utilisées** :
 - **Saisie avec scanf et validation des données** (ex : sexe uniquement M ou F).
 - **Utilisation d'une boucle for pour N entrées** afin d'éviter les erreurs de saisie multiples.
 - **Vérification de l'unicité** des prénoms pour éviter les doublons.

2. pv creer_voiture_manuel(int N)

- **Objectif** : Créer une liste chaînée de voitures avec saisie utilisateur.
- **Techniques utilisées** :
 - **Demande des informations véhicule par véhicule.**
 - **Vérification des doublons avec strcmp** sur les matricules.
 - **Insertion dans une liste triée** pour faciliter les recherches ultérieures.

3. pf creer_famille_manuel(int N)

- **Objectif** : Créer manuellement une liste de familles.
- **Techniques utilisées** :
 - **Appel des fonctions de création** `creer_personne_manuel` et `creer_voiture_manuel` pour remplir les sous-structures.
 - **Validation de la cohérence des données** avant d'ajouter la famille dans la liste.
 - **Chaînage correct des familles** pour assurer l'intégrité de la population.

Main.c

1. Module d'Interface Utilisateur

Rôle :

Gère l'affichage des menus et l'interaction avec l'utilisateur (clavier/souris).

Fonctions clés :

1. `detecte(int *cpt, int taille, bool *enter)`

- **Rôle** : Détecte les entrées clavier (flèches, Entrée) et souris.
- **Implémentation** :
 - Utilise `GetStdHandle(STD_INPUT_HANDLE)` pour capturer les événements.
 - Gère les touches `VK_UP`, `VK_DOWN`, `VK_RETURN`.
 - Détecte les clics souris via `MOUSE_EVENT_RECORD`.

2. `menu_principal(int *choix)`

- **Rôle** : Affiche le menu principal (Partie I, Partie II, Quitter).
- **Technique** :
 - Utilise des codes couleur ANSI (`GREEN`, `RED`, `RESET`).
 - Met en surbrillance l'option sélectionnée.

3. `menu_1(int *choix)` et `menu_2(int *choix)`

- **Rôle** : Affichent les sous-menus des Parties I et II.
- **Particularités** :
 - Navigation interactive avec `detecte()`.
 - Affichage dynamique des options.

2. Module de Gestion des Populations (Partie I)

Rôle :

Gère les opérations de base sur les populations de familles.

Fonctions principales :

1. `partie_1()`

- **Rôle** : Point d'entrée de la Partie I.
- **Fonctionnalités** :
 - Création de populations (`creer_pop`, `creer_pop_manuel`).
 - Tri (`trier_pop`), fusion (`fusion_pops`), suppression (`supp`).

2. `creer_pop(int val)`

- **Rôle** : Génère une population aléatoire.
- **Implémentation** :
 - Utilise `rand()` pour générer des données fictives.
 - Stocke dans une liste chaînée.

3. `trier_pop(struct famille1 *pop)`

- **Rôle** : Trie les familles par nom.
- **Algorithme** : Tri par insertion ou tri rapide (selon `part1.h`).

4. `fusion_pops(struct famille1 *pop1, struct famille1 *pop2)`

- **Rôle** : Fusionne deux populations.
- **Technique** : Concaténation de listes chaînées + suppression des doublons.

3. Module de Gestion Avancée (Partie II)

Rôle :

Offre des fonctionnalités avancées (statistiques, modifications fines, export).

Fonctions clés :

1. `partie_2()`

- **Rôle** : Point d'entrée de la Partie II.
- **Fonctionnalités** :

- Gestion des parents/enfants/voitures.
- Export de rapports (exporter_fichier).

2. **modifier_famille(pf famille, ch20 nomfam)**

- **Rôle** : Modifie une famille (ajout/suppression de membres).
- **Actions** :
 - Suppression de parents/enfants (supprimer_parent, supprimer_enfant).
 - Ajout de nouveau-nés (naissanceTrie).

3. **exporter_fichier(pf pop1, pf pop2)**

- **Rôle** : Exporte des rapports dans des fichiers texte.
- **Types de rapports** :
 - Statistiques (ecrit_rapport_pop).
 - Certificats de naissance (cert_naissance).
 - Rapports de vol (ecrit_rapport_vol).

4. Module de Confirmation et Saisie

Rôle :

Gère les dialogues de confirmation et les choix utilisateur.

Fonctions :

1. **confirmation()**

- **Rôle** : Demande une confirmation (Oui/Non).
- **Affichage** : Utilise des couleurs pour mettre en évidence les choix.

2. **methode()**

- **Rôle** : Demande à l'utilisateur de choisir entre mode manuel/automatique.
- **Interaction** : Utilise _getch() pour détecter les flèches.

5. Module Utilitaires

Rôle :

Fournit des fonctions helper pour les opérations courantes.

Fonctions :

1. **clear_screen()**
 - **Rôle** : Efface la console (via `system("cls")` ou codes ANSI).
 2. **moveCursorToTop()**
 - **Rôle** : Replace le curseur en haut de l'écran.
 3. **Color(int couleurDuTexte, int couleurDeFond)**
 - **Rôle** : Change les couleurs du texte (Windows API).
-

6. Structures de Données

Rôle :

Définit les structures pour représenter les familles.

Structures (déclarées dans `table.h`) :

1. **struct famille1** (Partie I)
 - Contient :
 - Nom de famille.
 - Liste chaînée de membres.
 2. **struct famille2** (Partie II)
 - Contient :
 - Parents (père, mère).
 - Enfants (liste chaînée).
 - Voitures (liste chaînée).
-

Techniques d'Implémentation Notables

1. **Gestion des Couleurs**
 - Macros comme `GREEN "\x1B[32m"` pour un code plus lisible.
2. **Navigation Souris/Clavier**
 - Utilisation de l'API Windows (`GetStdHandle`, `ReadConsoleInput`).
3. **Gestion Mémoire**
 - Allocation dynamique (`malloc`) pour les familles.

- Libération explicite (liberer_famille).

Conclusion

Ce fichier main.c est une application complète de gestion de populations familiales, avec :

- Une **interface interactive** (menus, couleurs, navigation).
- Deux **modules fonctionnels** (Partie I basique, Partie II avancée).
- Une **gestion mémoire rigoureuse** (allocations/libérations).
- Des **fonctions utilitaires** pour améliorer l'expérience utilisateur.

Conclusion et Perspectives d'Évolution

Ce projet **TP_ALSDD** a permis de mettre en œuvre avec succès un **système de gestion de population** en C basé sur des listes chaînées, offrant deux niveaux fonctionnels :

- **Gestion simplifiée** (Partie 1) avec tri, recherche et fusion
- **Gestion avancée** (Partie 2) intégrant parents, enfants et véhicules

Réalisations Clés

- ✓ **Gestion optimisée** de la mémoire avec allocation dynamique
- ✓ **Interface intuitive** avec navigation clavier/souris
- ✓ **Architecture modulaire** bien organisée (part1/part2/table)
- ✓ **Documentation complète** couvrant tous les aspects techniques

Vision pour l'Avenir

Nous ambitionnons d'étendre ce système vers :

1. **Une modélisation arborescente** pour représenter plusieurs générations
2. **L'ajout de données sociodémographiques** :
 - Profession, âge, revenu, localisation
 - Statistiques avancées (moyennes, répartitions)
3. **Des analyses plus poussées** :
 - Pyramides des âges
 - Mobilité géographique
 - Niveaux de vie

Contraintes actuelles :

- Le délai imparti ne permet pas cette extension immédiate

- L'évolution vers un arbre nécessiterait une refonte importante

Prochaines Étapes Potentielles

- Concevoir la structure arborescente (nœuds et relations)
- Adapter les algorithmes existants
- Développer les nouveaux modules d'analyse
- Optimiser les performances pour de grands jeux de données

Ce projet constitue une **base solide** pour des développements ultérieurs. Son architecture modulaire permettra une évolution progressive vers ces nouvelles fonctionnalités.

"Un programme bien conçu est comme un arbre généalogique : plus ses racines sont solides, plus il peut s'élever et s'étendre."